PRNN Programming Assignment 2

Arjun Mallick
Sr. 25989

1.1 Flattened Temporal MLP (Regression)

Data Preparation

A sliding window of 72 hours is used, flattening temporal features into a 576-dimensional vector. Standard scaling is applied for normalization.

Split	Input Shape	Target Shape
Train	(13047, 576)	(13047, 1)
Validation	(2721, 576)	(2721, 1)
Test	(2720, 576)	(2720, 1)

Table 1: Flattened dataset dimensions

Model

A 3-layer MLP ($576 \to 128 \to 64 \to 1$) with ReLU activations is trained using MSE loss.

Results and Discussion

Dataset	MSE	RMSE
Train	0.5084	0.7130
Validation	0.2183	0.4673
Test	0.8137	0.9020

Table 2: Model performance

The model performs well on validation but shows higher test error. Flattening removes temporal structure, limiting the model's ability to capture sequential dependencies.

1.2 The Vanishing Gradient Proof

Experimental Setup

The MLP is re-initialized with Sigmoid activations. Gradients of weights in all layers are captured during the first epoch using backward hooks, and their L2 norms are recorded per mini-batch.

Gradient Flow Visualization

Fig.1 shows the gradient norm for all three layers in the model.

Observation

Gradients are largest in the final layer and decrease progressively toward earlier layers, with the first layer having near-zero gradients. This indicates vanishing gradients.

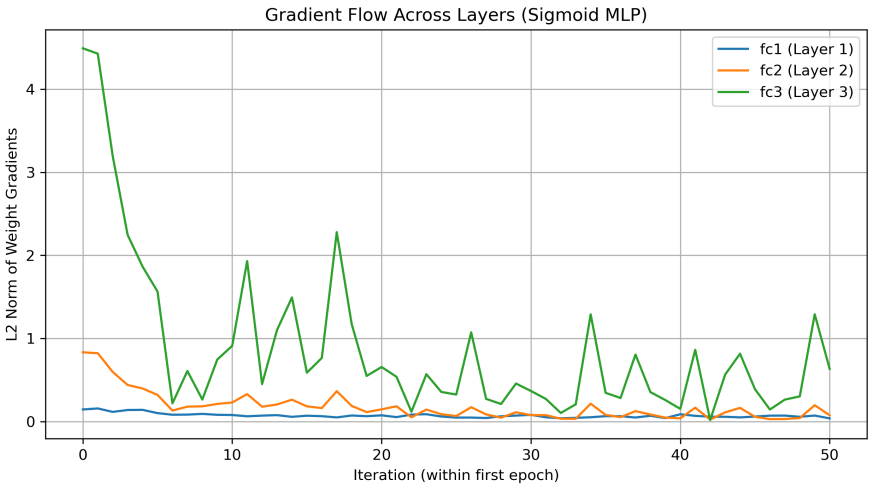

Figure 1: L2 norm of weight gradients across layers during the first epoch

Mathematical Explanation

For sigmoid activation:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)) \leq 0.25$$

Backpropagation gives:

$$\frac{\partial L}{\partial W^{(l)}} = \frac{\partial L}{\partial W^{(L)}} \prod_{k=l}^{L-1} \sigma'(z^{(k)})$$

Thus,

$$\frac{\partial L}{\partial W^{(1)}} \approx (0.25)^L \cdot \frac{\partial L}{\partial W^{(L)}}$$

Gradients decay exponentially for earlier layers, causing vanishing gradients.

1.3 Imbalanced Threshold Classification

Setup

The regression target is converted to a binary classification task based on PM2.5 value. The weights are chosen based on number of instances in each class pos_weight $= \frac{N_{neg}}{N_{pos}}$

Precision-Recall Curves

Fig.2 shows the resulting PR Curve for the two models

Observations

- The weighted loss increases the contribution of positive samples which leads to better precision at low recall and higher PR-AUC.
- Both models converge to similar performance at high recall. Weighted BCE improves performance under class imbalance, though the overall gain is moderate due to mild imbalance in the dataset.

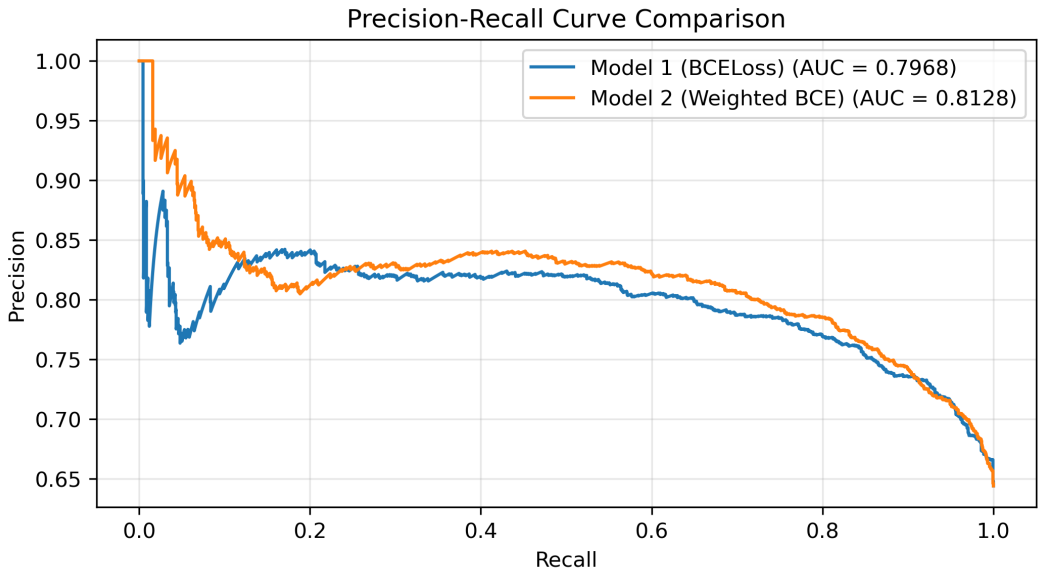

Figure 2: Precision-Recall curve comparison

2.4 CNN from Scratch & Receptive Fields

Architecture

The CNN consists of three sequential convolutional blocks. Each block follows the structure:

$$\text{Conv}(3 \times 3, s = 1) \rightarrow \text{BatchNorm} \rightarrow \text{ReLU} \rightarrow \text{MaxPool}(2 \times 2, s = 2)$$

The number of channels increases across blocks ($16 \rightarrow 32 \rightarrow 64$), enabling hierarchical feature extraction from low-level to high-level representations.

Feature Map Sizes

$$128 \times 128 \rightarrow 64 \times 64 \rightarrow 32 \times 32 \rightarrow 16 \times 16$$

Receptive Field Progression

$$3 \rightarrow 4 \rightarrow 8 \rightarrow 10 \rightarrow 18 \rightarrow 22$$

Final Output

$$\text{Feature Map Size} = 16 \times 16, \quad \text{Receptive Field} = 22 \times 22$$

Explanation:

Each convolution layer extracts local features while preserving spatial resolution, and max-pooling reduces spatial dimensions while increasing the receptive field. Stacking multiple such blocks allows the network to capture increasingly larger spatial context, with the final layer aggregating information from a 22×22 region of the input.

2.5 CNN Regression Head

Setup and Target Generation (Severity Percentage)

The CNN architecture was modified to perform regression by replacing the final layer with a single neuron that outputs a continuous *Severity Percentage*. The model is trained using Mean Squared Error (MSE) loss.

The severity target is defined as the ratio of diseased (necrotic) region to the total leaf region. Segmented images are used to extract leaf pixels by removing the background (non-black pixels). For each crop, healthy

images are used to estimate HSV statistics (mean and standard deviation), which define the distribution of healthy leaf color. Pixels deviating from this distribution are classified as diseased. Bounding boxes are computed for both the diseased region and the full leaf, and the severity is calculated as the ratio of their areas.

Model Setup

The network outputs a single continuous value corresponding to severity. The loss function used is Mean Squared Error (MSE), and performance is evaluated using Mean Absolute Error (MAE).

Results and Conclusion

- Train MAE: 0.015586
- Validation MAE: 0.015629
- Test MAE: 0.015626

The model achieves low and consistent MAE across all splits, indicating good generalization and effective learning of the severity estimation task.

2.6 Translation Invariance

Experiment Setup

A subset of 100 test images was evaluated. A translation of 5 pixels right and down (with zero padding) was applied, and the model was tested on both original and shifted images.

Results

Test Accuracy (Original): 0.94
Test Accuracy (Shifted): 0.78

Performance Drop and Explanation

The significant drop in accuracy shows sensitivity to spatial shifts. Max-pooling provides only approximate invariance, as it operates on fixed grids; small translations shift features across pooling regions, altering activations. Additionally, discrete sampling in convolution and zero padding further change feature responses, preventing perfect translation invariance.

3.7 Vanilla RNN from Scratch

Methodology

A vanilla RNN cell is implemented using `nn.Linear` and `tanh`, treating it as an MLP with memory. At each time step:

$$h_t = \tanh(W[x_t, h_{t-1}] + b)$$

where the input and previous hidden state are concatenated. The same weights are shared across all time steps, and a `for` loop is used to unroll the sequence. The final hidden state is passed through a linear layer to predict PM2.5.

Results and Discussion

Split	MSE	RMSE
Train	0.5941	0.7708
Validation	0.2176	0.4665
Test	0.7474	0.8645

The model captures temporal dependencies effectively. Lower validation error suggests stable generalization, while higher test error indicates possible distribution shift or limited model capacity. The vanilla RNN is constrained by issues such as vanishing gradients and absence of gating mechanisms.

3.8 BPTT Decay

Methodology

A 100-step sequence is passed through an untrained vanilla RNN. After computing the loss at the final time step, `.backward()` is applied. The gradient norms $\left\| \frac{\partial L}{\partial h_t} \right\|$ are extracted at $t = 100, 50, 0$.

Results

$$\textbf{t=100:} 1.6571 \times 10^{-1}, \quad \textbf{t=50:} 5.4033 \times 10^{-15}, \quad \textbf{t=0:} 0.0$$

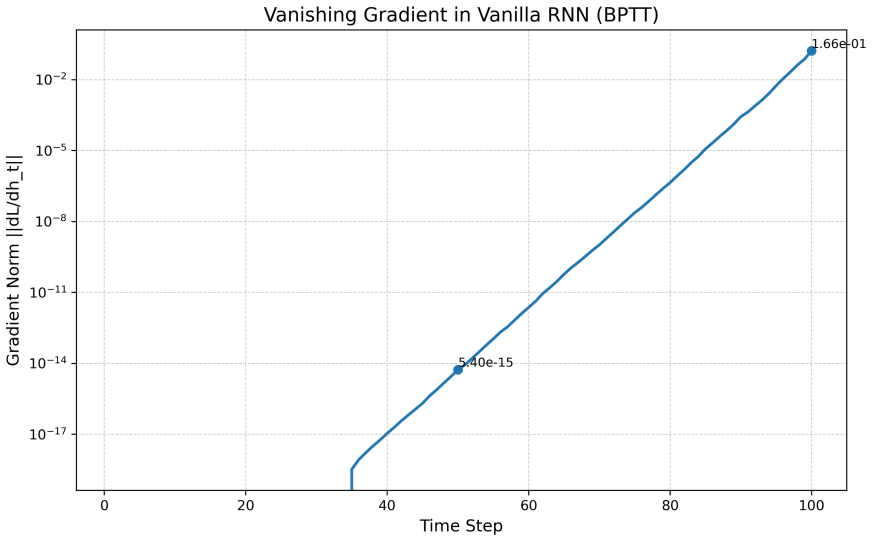

Figure 3: Gradient norm decay across time steps (log scale on y-axis)

Observation

The gradient magnitude decays rapidly as we move backward in time, becoming nearly zero at earlier steps. This demonstrates the vanishing gradient problem, where long-term dependencies cannot be effectively learned due to exponential decay of gradients during BPTT.

3.9 Exploding Gradients

Setup

The learning rate of the vanilla RNN was artificially increased and gradient clipping removed. Training was continued until the loss became NaN.

Results

Explosion occurred at epoch 14. Just before this:

$$\text{Val Loss} \approx 2.39 \times 10^{35}, \quad \|\nabla\| \approx 1.60 \times 10^{18}$$

The maximum singular value of the recurrent weight matrix W_h

$$\sigma_{\max}(W_h) \approx 1.366 \times 10^8$$

Discussion

During backpropagation through time (BPTT), gradients are propagated as a product of Jacobian matrices:

$$\frac{\partial L}{\partial h_t} = \prod_{k=t}^{T} \left(W_h^\top \cdot D_k\right)$$

where $D_k = \text{diag}(1 - h_k^2)$ and satisfies $\|D_k\| \leq 1$.

Taking norms, we obtain:

$$\left\| \frac{\partial L}{\partial h_t} \right\| \leq (\sigma_{\max}(W_h))^{T-t}$$

Since:

$$\sigma_{\max}(W_h) \approx 1.366 \times 10^8 \gg 1,$$

the gradient norm grows exponentially with the number of time steps:

$$\left\| \frac{\partial L}{\partial h_t} \right\| \sim (1.366 \times 10^8)^{T-t}$$

This leads to extremely large gradient values, causing numerical overflow during training. As a result, the loss diverges to Inf and eventually becomes NaN, as observed in the experiment.

4.10 LSTM Gradient Rescue

Results

Figure 4 compares gradient norms.

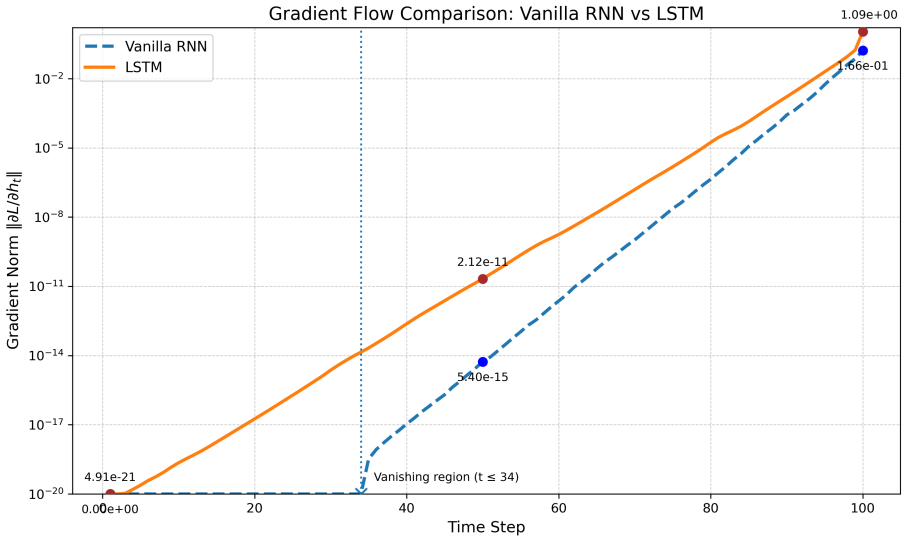

Figure 4: Gradient flow: Vanilla RNN vs LSTM

- RNN gradients vanish ($\sim 10^{-20}$ at early steps $t \approx 35$).
- LSTM maintains larger gradients across time.
- At $t = 50$: 2.12×10^{-11} (LSTM) vs 5.40×10^{-15} (RNN).
- At $t = 0$: RNN ≈ 0, LSTM remains non-zero. (As shown in the fig. 4)

Explanation

The LSTM cell update

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \tag{1}$$

provides a linear gradient path:

$$\frac{\partial L}{\partial c_{t-1}} = \frac{\partial L}{\partial c_t} \cdot f_t \tag{2}$$

When $f_t \approx 1$, gradients do not decay. Thus, the **forget gate** prevents vanishing.

4.11 GRU vs. LSTM Efficiency

Setup

We train LSTM and GRU models on the same regression task and compute the number of trainable parameters using the model's parameters. We compare validation MSE and training time per epoch.

Results

- **Parameter Count:** LSTM = 19009, GRU = 14273
- **Validation MSE:** LSTM = 0.3095, GRU = 0.2971
- **Time/Epoch:** LSTM = 0.67s, GRU = 0.46s

Conclusion

GRU has fewer parameters due to fewer gates, resulting in faster training. It achieves slightly better validation performance in this task, indicating a more efficient trade-off between complexity and accuracy.

4.12 Sequence-to-Sequence Forecasting

Setup

An encoder-decoder LSTM is used where the encoder processes 72-hour history into a context vector (h, c), and an autoregressive decoder predicts the next 24 time steps.

Results

Figure 5 shows the predicted 24-step trajectory against the ground truth.

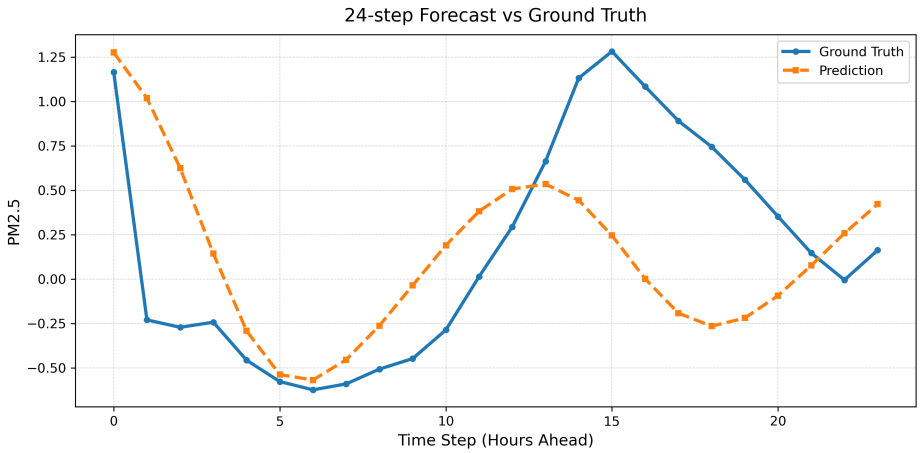

Figure 5: 24-step forecast vs ground truth

Observation

The model captures the overall trend of the sequence but fails to accurately model sharp peaks and variations.This shows that prediction error increases over time due to autoregressive decoding.

5.13 Scaled Dot-Product Attention

Methodology

A single-head scaled dot-product attention mechanism was implemented from scratch using PyTorch tensor operations. Given an input sequence $X \in \mathbb{R}^{72\times d}$, the query, key, and value matrices were computed as:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

where W_Q, W_K, W_V are learnable projection matrices.

The attention weights were computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

The model was trained for a regression task using the 72-hour meteorological time series data. The attention weight matrix of size 72×72 was extracted for a test sample and visualized.

Result and Discussion

The attention weight matrix for a representative test sample is shown in Figure 6. The heatmap shows structured, non-uniform attention patterns, indicating that the model captures meaningful temporal dependencies. Periodic grid-like patterns suggest the presence of recurring temporal behavior in the data.

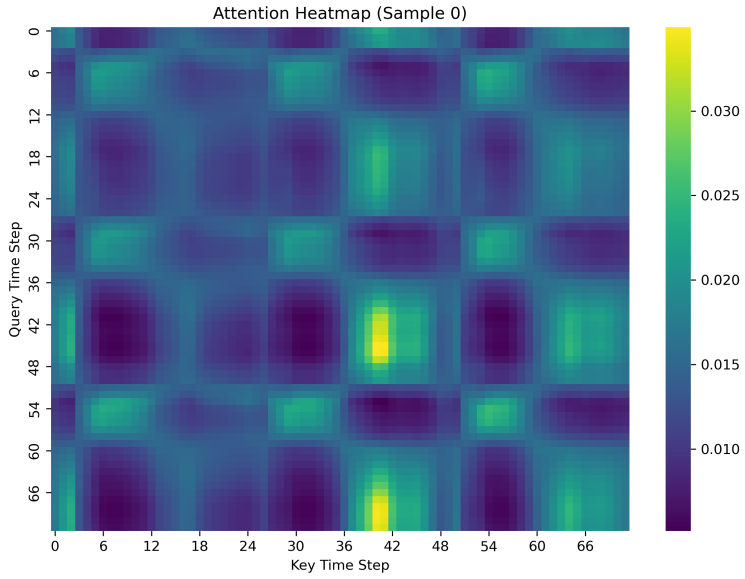

Figure 6: Attention heatmap for a test sample

5.14 The Necessity of Positional Encoding

Methodology and Results

A Transformer encoder was trained on the time-series data without positional encoding, followed by a second model where standard sinusoidal positional encodings were added to the input embeddings. Both models were trained under identical settings, and their validation losses were compared. The Validation Losses are:

- Validation Loss (without positional encoding): 0.2670
- Validation Loss (with positional encoding): 0.2571

The lower validation loss with positional encoding indicates improved learning of temporal structure. Without positional encoding, the model fails to capture ordering information inherent in time-series data.

Mathematical Explanation

The self-attention mechanism operates on input embeddings $X = [x_1, x_2, \ldots, x_n]$ by computing:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

and

$$\text{Attention}(X) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Let $P \in \mathbb{R}^{n \times n}$ be any permutation matrix. Then, for a permuted input PX:

$$Q' = (PX)W_Q = PQ, \quad K' = PK, \quad V' = PV$$

The attention scores become:

$$Q'K'^T = (PQ)(PK)^T = PQK^TP^T$$

Applying softmax row-wise preserves permutation structure:

$$\text{softmax}(PAP^T) = P \cdot \text{softmax}(A) \cdot P^T$$

Thus,

$$\text{Attention}(PX) = P \cdot \text{Attention}(X)$$

This shows that self-attention is *permutation equivariant*, i.e., permuting the input sequence results in the same permutation of the output. Consequently, the model has no inherent notion of order and cannot distinguish between different temporal arrangements of the same set of inputs.

With positional encoding, each input is modified as:

$$x_i \to x_i + p_i$$

where p_i is a deterministic function of position. Since $p_i \neq p_j$ for $i \neq j$, any permutation P changes both the values and their associated positional information:

$$P(x_i + p_i) \neq x_j + p_j$$

Thus, positional encodings break permutation equivariance and introduce explicit order information, enabling the model to capture temporal dependencies.

5.15 Vision Transformer (ViT)

Parameter Efficiency Comparison

The parameter efficiency of the CNN (from Phase 2) and the Vision Transformer (ViT) was evaluated in terms of test accuracy and number of learnable parameters, as shown in Table 3.

Model	Test Accuracy(%)	# Parameters	Accuracy/Parameter
CNN	94.78	408,294	2.0×10^{-4}
ViT	93.51	335,270	3.0×10^{-4}

Table 3: Comparison of parameter efficiency between CNN and ViT

Discussion

From Table 3, the CNN achieves higher test accuracy, indicating better absolute performance. However, the ViT exhibits higher accuracy per parameter, making it more parameter efficient. This highlights that while CNNs leverage strong inductive biases for image tasks, the ViT utilizes its parameters more effectively.

6.16 Transfer Learning & Freezing

Approach:

A pre-trained `ResNet18` model (trained on ImageNet) is used as a feature extractor. All convolutional layers are frozen to retain learned low-level and mid-level visual features. The final fully connected layer is replaced with a new linear layer mapping to 5 output classes. Only this classification head is trained.

Parameter Analysis

- Trainable Parameters: 2,565
- Frozen Parameters: 11,176,512

This shows that only a very small fraction of the network is updated during training, significantly reducing computational cost and risk of overfitting.

Validation Loss Curve

Freezing the backbone and training only the final layer provides a stable and efficient learning setup. The model successfully adapts to the target task while avoiding overfitting and excessive parameter updates.

Figure 7: Training and Validation Loss over Epochs

6.17 Ordinal Imbalance & Focal Loss

Approach

The labels (0–4) are ordinal and highly imbalanced toward Class 0. To address this, Focal Loss is used:

$$\mathcal{L}_{focal} = -(1 - p_t)^{\gamma} \log(p_t)$$

which down-weights easy samples ($p_t \to 1$) and focuses learning on hard, minority class examples.

Evaluation (QWK)

Performance is measured using Quadratic Weighted Kappa:

$$\kappa = 1 - \frac{\sum_{i,j} W_{ij} O_{ij}}{\sum_{i,j} W_{ij} E_{ij}}, \quad W_{ij} = \frac{(i-j)^2}{(K-1)^2}$$

which penalizes predictions proportionally to their ordinal distance.

Results

- Train: Loss = 0.0507, Acc = 0.7214, QWK = 0.8040
- Val: Loss = 0.0577, Acc = 0.7195, QWK = 0.7918

Validation Loss Curve:

Why Accuracy is Misleading

Accuracy,

$$\text{Accuracy} = \frac{1}{N} \sum_i \mathbf{1}\{y_i = \hat{y}_i\},$$

treats all errors equally and is biased toward the majority class. In this dataset, predicting Class 0 frequently yields high accuracy despite poor minority class performance. Moreover, it ignores ordinal structure, assigning equal penalty to all misclassifications.

In contrast, QWK incorporates both class imbalance (via E) and ordinal distance (via W_{ij}), making it a more appropriate metric. Thus, Focal Loss improves minority class learning, and QWK provides a reliable evaluation aligned with the ordinal nature of the task.